BECM36MLM: Machine Learning Methods

Week 2: Learning from **Relational** Data

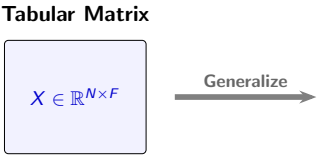
Gustav Šír, PhD

Outline

The Single-Table Illusion vs. Relational Reality

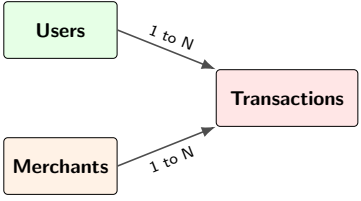
Week 1: The Flat Grid

- Data: Matrix $X \in \mathbb{R}^{N \times F}$
- Assumption: "Flat-World" (independent rows)



Week 2: The Relational Schema

- Data: Database $D = \{T_1, T_2, \ldots\}$
- Links: Connected by Foreign Keys
- Assumption: "Structured-World" (interacting entities)



The Fall of the I.I.D. Assumption

Standard ML Cornerstone

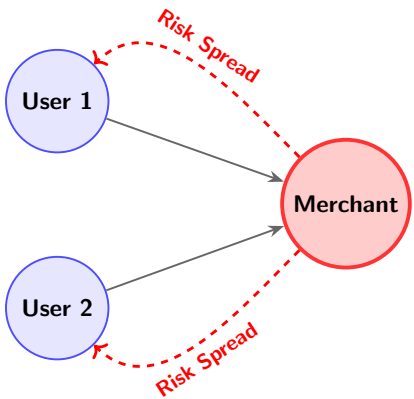
- **I.I.D.** (Independent & Identically Distributed)
- $P(y_i \mid x_i) \perp P(y_j \mid x_j)$

The Relational Violation

- DBs link rows by design, breaking independence.

Relational Autocorrelation

- A node's label predicts its neighbors' labels.
- Ex: Fraud probability spikes for shared merchants.



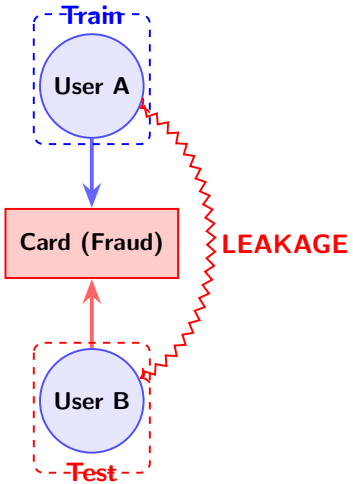
The Danger of Structural Leakage

Q: Engagement Question

User A and B share a fraudulent credit card.
A is in Train, B is in Test.
What happens to your model's accuracy?

The Topology of Leakage

- **Illusion:** Flawless test metrics, production failure.
- **Memorization:** Model learns the shared *identity* (the card).
- **Failure:** Fails to learn the *general pattern* of fraud.



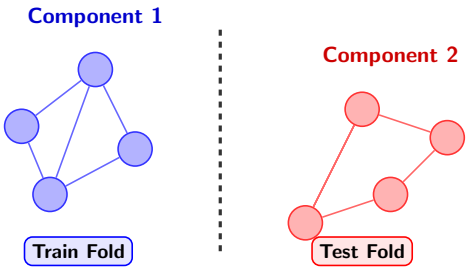
Restoring Integrity: Block & Group Splitting

Group Splitting (Entity-Level)

- Abandon random K-Fold cross-validation!
- Group dependent rows (e.g., all of User A's data) into one fold.

Connected Components Splitting

- Highly connected data requires disjoint subgraphs.
- NP-hard combinatorial optimization to minimize similarity-induced leakage (e.g., *DataSAIL Framework:*).



The Impedance Mismatch: Sets vs. Vectors

1. Relational DBs

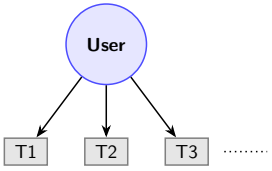
- Entities map to variable-sized *sets*.
- No fixed cardinality, no inherent order.

2. Machine Learning

- Requires fixed-length vector $x \in \mathbb{R}^F$.
- Math operates on fixed dimensions.

3. The Challenge

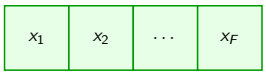
- **Mismatch:** How to feed dynamic sets into fixed vector math?



Dynamic Set (DB)

VS

Impedance Mismatch



Fixed Vector $\mathbb{R}^F$ (ML)

Propositionalization: The "Classical" Hack

1. The Mismatch Solution

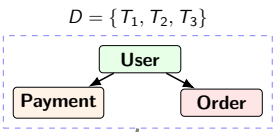
- How to feed a multi-table DB into XGBoost?
- **Propositionalization:** Squeezing relations into a flat table.

2. The Mathematical Goal

- Map Database D to Matrix $X \in \mathbb{R}^{N \times F}$.
- Decouples feature from model construction.

3. The "Classical" Hack

- Allows immediate use of Week 1 models (Random Forest, SVM).



User_ID	Age	Num_Orders	Avg_Payment
U1	34	12	$45.50
U2	28	3	$120.00

Matrix $X \in \mathbb{R}^{N \times F}$

The Mechanics of Flattening: Aggregation

Handling 1-to-Many Relations

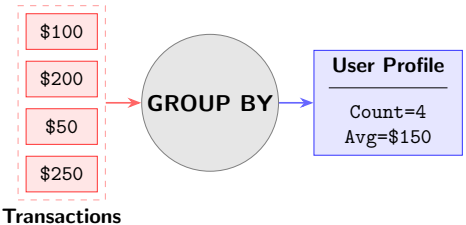
- Target entity has a dynamic multi-set of child records.
- **Solution:** Summarize sets into scalar values.

Aggregation Types

- **Distributive:** `MIN`, `MAX`, `SUM`, `COUNT`.
- **Algebraic:** `AVG`, `VARIANCE`.
- **Existential:** Binary flags (`Has_Foreign_Tx = 1`).

Example (SQL equivalent)

```
SELECT u.ID, COUNT(t.ID),
AVG(t.Amount)
FROM Users u JOIN Transactions t
ON ...
GROUP BY u.ID
```



Deep Feature Synthesis (DFS)

1. Automating the Hack

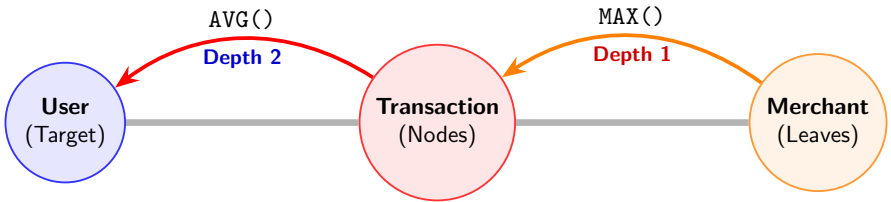
- Manual aggregation is slow and biased.
- **DFS:** Automates feature generation over an *EntitySet*.

2. Stacking Primitives

- Recursively applies mathematical aggregates (e.g., MIN, MAX) over relations.

3. The Concept of Depth

- *Depth 1:* `MAX(Tx.Amt)`
- *Depth 2:* `AVG(Merch.MAX(Tx))`
- Captures multi-hop links.



From Features to Rules (Propositional Logic)

The Propositional Rule

- Aggregated features act as **logical literals**.
- `IF COUNT(Tx) > 50`
  `AND AVG(Amt) > 1000`
  `THEN Fraud`

Can we Build Explicit Rules?

- Trees hide these in paths!
- We might also want explicit, isolated, interpretable logic for high-stakes domains (finance, medicine).

RIPPER (Sequential Covering)

- Greedily grows a rule to high purity.
- Prunes back using a validation set to ensure generalization.
- Classic, highly efficient logic miner.

Skope-Rules (Ensemble Extraction)

- Trains a massive Random Forest.
- Extracts all paths from root to leaf as candidate rules.
- Filters and deduplicates keeping only high-precision rules.

The Limit: Topological Blindness

The Information Bottleneck

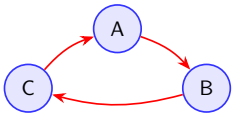
- Reducing a complex graph to summary statistics destroys topology.
- The flat table is "blind" to true network structure.

What We Lose

- **Cycles:** Flat rows cannot capture circular money flows ($A \to B \to C \to A$).
- **Paths:** Specific event sequences are washed out by `AVG()` and `SUM()`.

The Consequence

- If propositionalization destroys the signal, no algorithm (XGBoost, RIPPER) can recover it.



Money Laundering Ring

User_A	Sum=$5k	Avg=$1k

Fails!

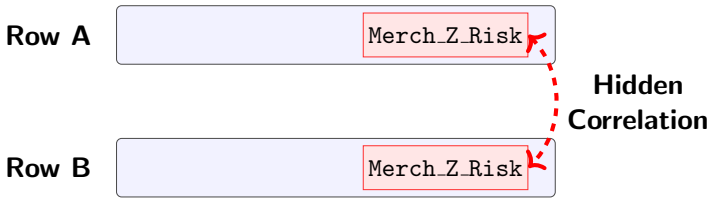
Engagement Question & The Lingering I.I.D Problem

Q: Engagement Question

If User A and User B share a risky Merchant, and we use DFS to aggregate the Merchant's risk into both Users' rows... are the two resulting feature vectors independent?

The Illusion of Independence

- **Answer:** NO. The rows are mathematically correlated.
- **Reality:** Flattening hides the I.I.D. violation; it doesn't solve it. We need ML to understand structure!



The Language of Structure

The Limit of Propositionalization

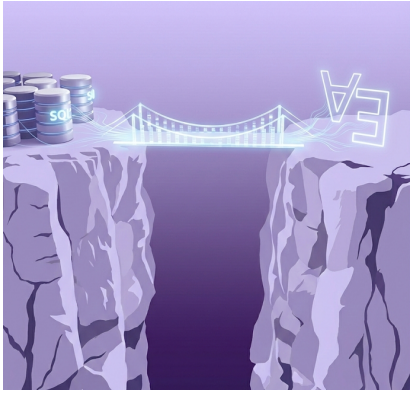
- Manual aggregation is static and loses topology.
- We need a dynamic language for *relationships*.

A Language You Already Know

- Relational databases run on **SQL**.
- SQL describes entity interactions natively.

The Revelation

- SQL is a practical implementation of **Predicate Logic**.
- We will treat DB exploration as automated theorem proving.



The SQL-Logic Analogy

The Mapping Analogy:

SELECT	$\Longleftrightarrow$	**Free Variables / Atoms**	*(Target output)*
FROM	$\Longleftrightarrow$	**Predicates / Relations**	*(The Domain)*
WHERE	$\Longleftrightarrow$	**Constraints / Filters**	*(Boundaries)*
JOIN	$\Longleftrightarrow$	**Conjunction ($\wedge$)**	*(Multi-hop dependencies)*

Conjunctive Queries (CQs)

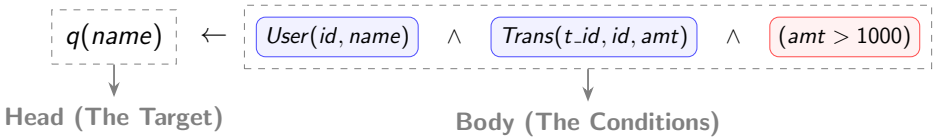
1. The SQL Query

Example Query

```
SELECT U.Name
FROM Users U JOIN Trans T ON U.ID = T.UID
WHERE T.Amount > 1000
```

2. The Logical Equivalent (Datalog Notation)

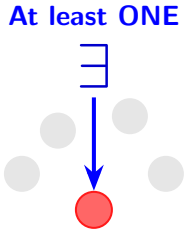
- **Query:** $q(name) \leftarrow User(id, name) \wedge Trans(t\_id, id, amt) \wedge (amt > 1000)$
- A **Conjunctive Query (CQ)** contains no disjunction (OR) and no negation.



Quantification: Existence and Grouping

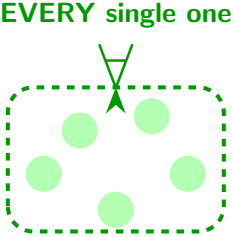
Existential Quantifier ($\exists$)

- **Meaning:** "There exists at least one..."
- **SQL:** `EXISTS (SELECT 1 ...)`
- **Usage:** Checking if a specific relational neighbor is present.



Universal Quantifier ($\forall$)

- **Meaning:** "For all..."
- **SQL:** `GROUP BY` / `HAVING COUNT(*) = Total`
- **Usage:** Summarizing entire relational sets.



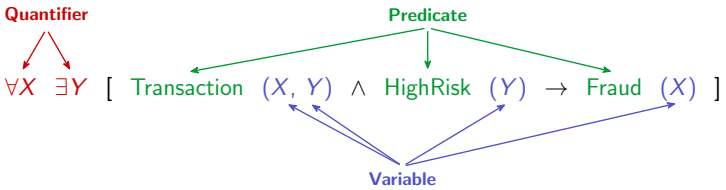
Formalizing First-Order Logic (FOL)

The Vocabulary of FOL

- **Constants:** Specific entities (*Alice*, *Merchant_Z*)
- **Variables:** Placeholders for entities (X, Y, Z)
- **Predicates:** Relations or properties (*Parent*(X, Y), *Fraud*(X))
- **Functions:** Mappings from objects to objects.

Example Formula

- *"For all X, if X has a transaction with a HighRisk Y, then X is Fraud."*



Horn Clauses: The Relational Template

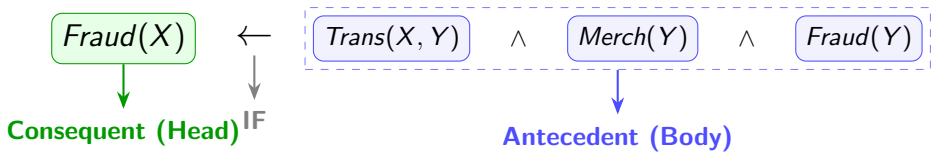
What is a Horn Clause?

- A logical rule containing **at most one positive literal**.
- The fundamental template for expressing relational patterns.

The Implication Syntax

- $Head \leftarrow Body_1 \wedge Body_2 \wedge \cdots \wedge Body_n$
- *Meaning:* "Head is true IF Body_1 AND Body_2... are true."

Relational Example:



Engagement Question: Translating Logic

Q: Engagement Question

If the SQL clause `WHERE Age > 18` acts as a filter, how would you represent this as a logical predicate in a Horn Clause predicting `CanVote(X)`?

A: The Logical Translation

$$CanVote(X) \leftarrow Person(X, Age) \wedge (Age > 18)$$

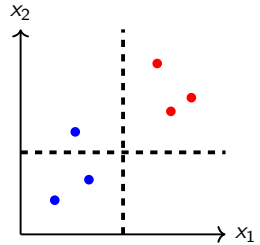
The Insight:

- Continuous math constraints ($Age > 18$) live alongside discrete structural logic ($\wedge$).
- We don't lose numerical feature engineering; we augment it.

The Paradigm Shift: The New Hypothesis Space

Week 1: Parameter Search

- **Space:** Axis-aligned cuts in Euclidean vector space.
- **Learning:** Finding best *numbers* to split a flat grid.



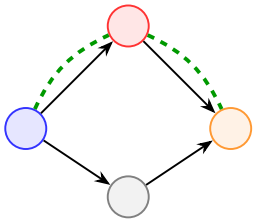
Vector Splits (Trees)

Shift

Week 2: Structure Search

- **Space:** Sets of First-Order Horn Clauses.
- **Learning:** Finding best *rules* (topology).

$$H(X) \leftarrow B_1(X, Y) \wedge B_2(Y)$$



Structural Motifs (Logic)

Inductive Logic Programming (ILP)

The Intersection

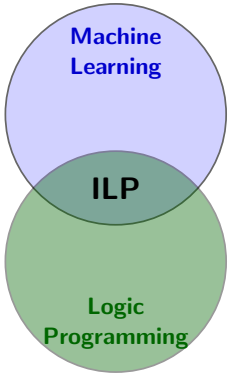
- ML automates induction (learning rules from data).
- **ILP:** Machine Learning $\cap$ Logic Programming.

The Paradigm Shift

- **Standard ML:** Learns *functions* from *tables*.
- **ILP:** Learns *logic programs* from *relations*.

The Core Components

- **Background Knowledge (B):** Known logical facts.
- **Examples (E^+, E^-):** Positive and negative instances.
- **Hypothesis (H):** The target logical rule to mine.



$$B \cup H \models E^+$$

The Learning from Entailment (LFE) Setting

The LFE Problem Definition

- **Given:**
  - Background Knowledge (B)
  - Positive Examples (E^+)
  - Negative Examples (E^-)
- **Find:** A Hypothesis H (a set of Horn clauses) that perfectly separates the examples.

1. Completeness (Covering Positives)

$$\forall e \in E^+ : B \cup H \models e$$

(Hypothesis + Background deduces all true examples)

2. Consistency (Excluding Negatives)

$$\forall e \in E^- : B \cup H \not\models e$$

(Hypothesis + Background does NOT deduce false examples)

Structuring the Hypothesis Space (θ-subsumption)

The Search Space

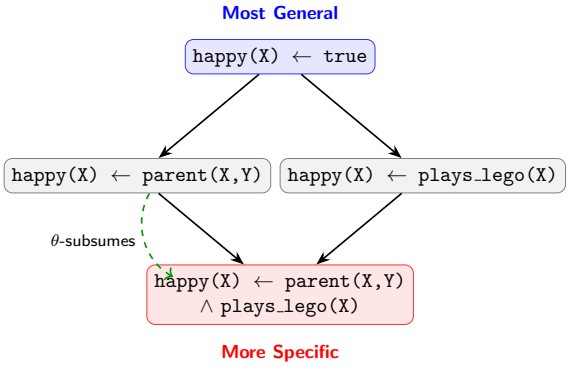
- The space of all possible logic rules is enormous!
- We must structure the space to search it algorithmically.

θ-Subsumption (Generality)

- Clause C_1 **subsumes** C_2 ($C_1 \preceq C_2$) if $\exists$ substitution θ such that $C_1\theta \subseteq C_2$.
- C_1 is *more general* than C_2.

Example

- $C_1 = f(A, B) \leftarrow head(A, B)$
- $C_2 = f(X, Y) \leftarrow head(X, Y) \wedge odd(Y)$
- C_1 subsumes C_2 using $\theta = \{A/X, B/Y\}$.



The FOIL Algorithm (Upgrading CART)

FOIL (First-Order Inductive Learner)

- Developed by Quinlan (1990) as a relational upgrade to Decision Trees.
- **Strategy:** Top-down, greedy separate-and-conquer.

The Target Rule Structure

- $Target(X_1, \ldots, X_m) \leftarrow L_1 \wedge L_2 \wedge \cdots \wedge L_n$

Algorithm Sketch

```
learnedRules = {}
while positive examples remain uncovered:
    rule = [Target(X...)  ← true]
    while rule covers negative examples:
       L = find_best_literal_to_add(rule)
       rule = rule ∧ L
    add rule to learnedRules
    remove covered positive examples
```

The Mechanics of FOIL (Information Gain)

Scoring Literals

- How do we choose the "best" literal L_i?
- **FOIL Information Gain:**

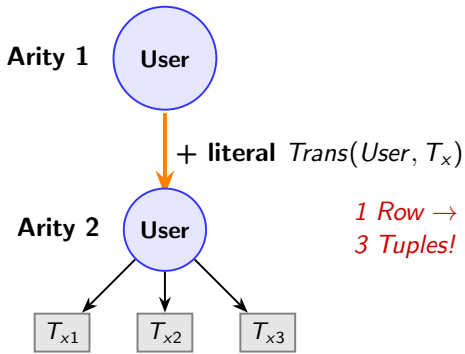
$$Gain(L_i) = p_{++} \times \left(\log_2 \frac{p'}{p'+n'} - \log_2 \frac{p}{p+n} \right)$$

Dynamic Tuples

- p, n = current positive/negative *tuples* covered.
- p', n' = tuples covered *after* adding L_i.
- p_{++} = positive tuples covered by *both*.

Determinate Literals

- Adding $Trans(User, T_x)$ introduces a *new variable* T_x.
- Changes tuple arity (size) dynamically!



Progol & Inverse Entailment

The Problem with Top-Down

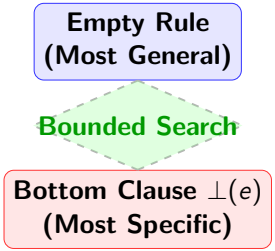
- FOIL's unconstrained top-down search space is astronomically large.
- **Solution:** Progol (Muggleton, 1995) uses **Mode-Directed Inverse Entailment (MDIE)**.

Step 1: The Seed & Bottom Clause

- Pick a specific positive example e.
- Construct the **Bottom Clause** $\perp(e)$: The most specific, maximally long rule explaining e.

Step 2: Bounded A* Search

- The hypothesis space is strictly bounded: *Empty* Rule $\succeq H \succeq \perp(e)$
- Search top-down *only* within these bounds.



Predicate Invention (PI)

The Background Knowledge Bottleneck

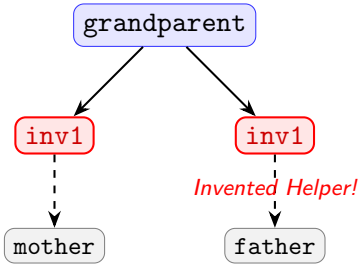
- ILP systems can only construct rules using predicates *already provided* in B.
- What if the necessary concept is missing?

What is Predicate Invention?

- The system automatically invents new, auxiliary predicate symbols.
- Equivalent to a human programmer writing a new "helper function".

Example: Learning `grandparent`

- Given: `mother`, `father`. Missing: `parent`.
- ILP invents `inv1(A,B)` = mother or father.
- $grandparent \leftarrow inv1(A, C) \wedge inv1(C, B)$



Upgrading the Tree: From CART to TILDE

Week 1: Propositional Trees (CART)

- Nodes test **attributes** of a fixed vector.
- Ex: `Age > 25` $\rightarrow$ True / False.
- Blind to relationships.

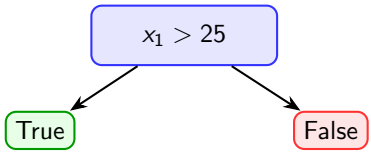
Week 2: Relational Trees (TILDE)

- Nodes test **logical queries** (existence/relations).
- Ex: $\exists Y : Trans(X, Y) \wedge (Amt(Y) > 1000)$.
- Navigates the graph topology directly.

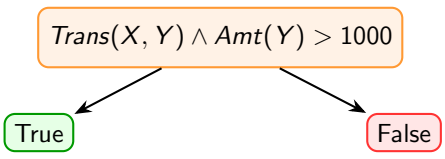
TILDE Overview

- **T**op-down **I**nduction of **L**ogical **D**ecision **Tree**s.
- Upgrades C4.5 to First-Order Logic.

CART (Tabular)



TILDE (Relational)



Learning from Interpretations (LFI)

The LFI Setting

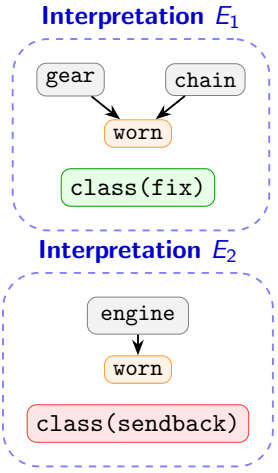
- Examples are not flat rows; they are **Interpretations** (worlds).
- An interpretation is a set of true facts describing an isolated graph.

Example: The Machine Repair Domain

- $E_1 = \{worn(gear), worn(chain), class(fix)\}$
- $E_2 = \{worn(engine), class(sendback)\}$

The Learning Task

- Given a set of interpretations E and Background Knowledge B.
- Learn a hypothesis H (a tree) predicting the *class*.



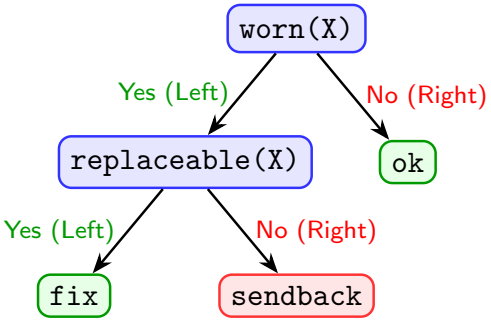
First-Order Logical Decision Trees (FOLDT)

The Node Structure

- Internal nodes contain a conjunction of literals.
- Left Branch = Query **Succeeds** (True).
- Right Branch = Query **Fails** (False).

Evaluating an Example (E)

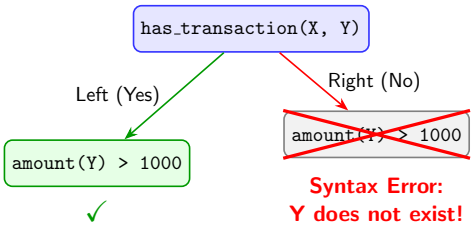
- To sort E, we run the node's query against $E \cup B$.
- A path from root to leaf represents a composite logical query.



Variable Scope & The Right Branch Restriction

Q: Engagement Question

If a node asks `has_transaction(X, Y)`, the Left branch means "Yes". The Right branch means "No". Can I ask `amount(Y) > 1000` on the Right branch?



The Right-Branch Restriction

- **Logic:** Variables introduced in a node are **existentially quantified** ($\exists Y$).
- **Left Branch:** The query succeeded. Y exists and is bound to data.
- **Right Branch:** The query failed. **"There is no such Y"**.
- **Rule:** Variables introduced in a node *cannot* occur in its right branch.

The TILDE Algorithm (Relational Splitting)

Greedy Top-Down Induction

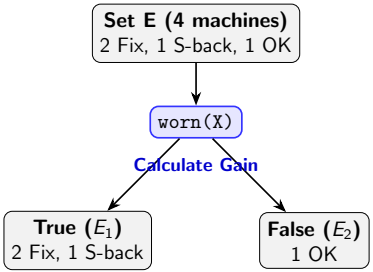
- Starts with an empty tree.
- Evaluates candidate literals from the *Language Bias*.

Information Gain on Tuples

- $Gain = Entropy(Parent) - \sum \frac{|E_i|}{|E|} Entropy(Child_i)$
- Evaluates splits by dividing interpretations E.

The Lookahead Solution

- **Issue:** Adding `Trans(X,Y)` alone might yield 0 gain.
- **Lookahead:** Perform multiple refinement steps at once (e.g., add `Trans` AND `Amount>1000`) to discover hidden gains.



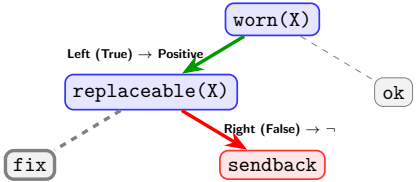
Extracting Logic Programs from FOLDTs

Trees to Rules

- Any tree can be converted to rules.
- In TILDE, every path from root to leaf is a **Horn Clause**.

Translation Mechanics

- Left Branch (True) $\rightarrow$ Add literal.
- Right Branch (False) $\rightarrow$ Add $\neg$ (negation).



Extracted Horn Clause

$$class(sendback) \leftarrow worn(X) \wedge \neg replaceable(X)$$

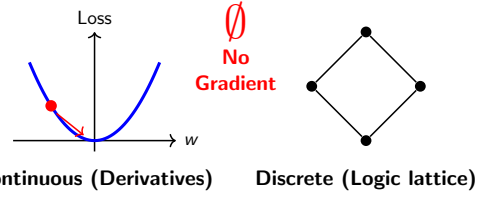
Engagement Question & The Need for Gradients

Q: Engagement Question

In Week 1, we used Gradient Descent to optimize our ML models. Why can't we just use Gradient Descent to search the ILP θ-subsumption lattice?

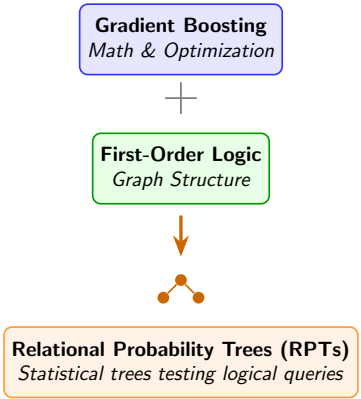
The Discrete Barrier

- **The Answer:** The lattice of logic rules is strictly **discrete**.
- You cannot take the derivative of a discrete logical literal ($Age > 20$ or $\exists$ Transaction).



The Grand Synthesis: Relational Boosting

1. **Week 1: GBMs**
   - **Strength:** Unmatched statistical optimization.
   - **Weakness:** Topologically blind (needs flat vectors).
2. **Week 2: Logic (ILP)**
   - **Strength:** Natively navigates multi-table graphs.
   - **Weakness:** Poor at handling noise and probabilities.
3. **The Climax: RDN-Boost**
   - **Synthesis:** Gradient Descent + Relational Logic.
   - **Result:** Boosting logical trees directly on DBs.



Relational Dependency Networks (RDNs)

The Problem with Bayesian Networks

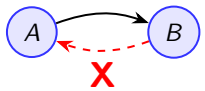
- Standard Bayesian Nets require Directed Acyclic Graphs (DAGs).
- **Relational Reality:** Data has cycles! (e.g., User A influences User B, and B influences A).

Dependency Networks

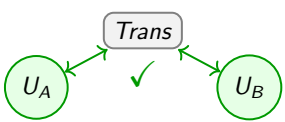
- Drops the acyclic requirement.
- Models the joint distribution via **pseudo-likelihood**.

Relational Dependency Networks (RDNs)

- Upgrades Dependency Nets to Relational Logic.
- Uses relational templates for dependencies.



Bayesian Network
(No Cycles Allowed)



RDN
(Valid Pseudo-Likelihood)

Relational Probability Trees (RPTs) as Weak Learners

The Weak Learner Upgrade

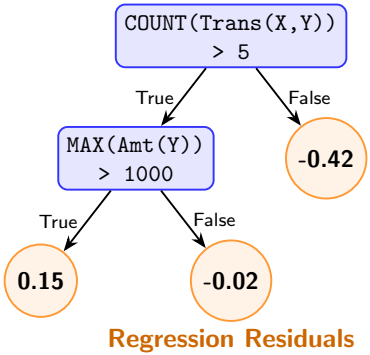
- **GBM:** Uses CART trees.
- **RDN-Boost:** Uses Relational Probability Trees (RPTs).

Inside the RPT

- Nodes = Relational Logic Queries.
- Leaves = Continuous values (Regression fits).

Dynamic Aggregation

- Nodes dynamically aggregate subsets: `AVG(Amt) > 1000`.



Functional Gradient Boosting for Relations

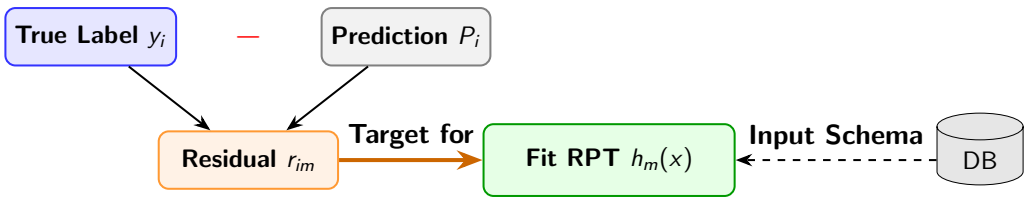
Gradient Descent in Function Space

- We seek $F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$ to minimize Log-Loss.
- $h_m(x)$ is a **function** (an RPT), not a parameter vector!

Computing Pseudo-Residuals

- For each example x_i, compute the error:
- $r_{im} = y_i - P(y_i = 1 \mid x_i, F_{m-1})$

The Relational Update Rule



The RDN-Boost Algorithm

Pseudocode

Algorithm RDN-Boost(Database D, Target y, Iterations M)

Initialize $F_0(x) = \log(P(y = 1)/P(y = 0))$

For $m = 1$ **to** M:

// 1. Calculate Gradients

For each example x_i **in** D:

$P_i = \text{Sigmoid}(F_{m-1}(x_i))$

$r_{im} = y_i - P_i$ // The Pseudo-Residual

// 2. Fit Weak Learner

Let $h_m =$ **Learn_RPT**(Database D, residuals r_m)

// 3. Update Ensemble

$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

Return F_M

The Core Mechanism: Lazy Propositionalization

Eager Propositionalization (DFS)

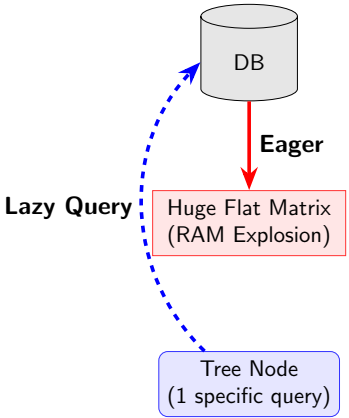
- **When:** *Before* training begins.
- **How:** Generate thousands of flat columns indiscriminately.
- **Flaw:** Massive memory explosion; irrelevant features.

Lazy Propositionalization (RDN-Boost)

- **When:** *During* tree induction (at node split).
- **How:** Construct relational queries dynamically as needed.

The Efficiency Shift

- Features are "invented" on the fly via Information Gain.
- Replaces the memory bottleneck with a query operation.



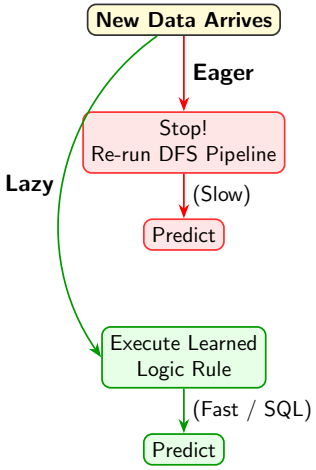
Engagement Question: Handling Dynamic Graphs

Q: Engagement Question

Imagine you use Eager Propositionalization (DFS) to train a model. Tomorrow, User A makes 50 new transactions, creating a new circular fraud pattern. How do you update User A's prediction?

A: The Re-computation Trap

- **Eager/DFS:** You must completely re-run the massive feature synthesis pipeline to update the flat matrix before predicting.
- **Lazy (RDN-Boost):** The logical rule $Fraud(X) \leftarrow Trans(X, Y) \ldots$ is already learned. You simply execute the rule as a fast SQL query on the updated DB!



Evaluating SRL Models (The Evaluation Crisis)

The Class Imbalance Problem

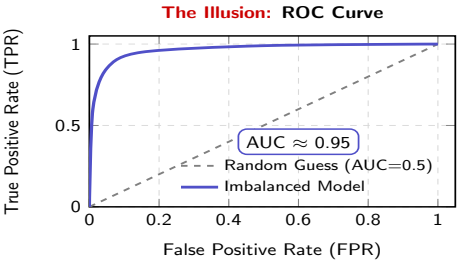
- In relational graphs, negatives vastly outnumber positives ($O(N^2)$ possible links).
- ROC-AUC becomes highly misleading.

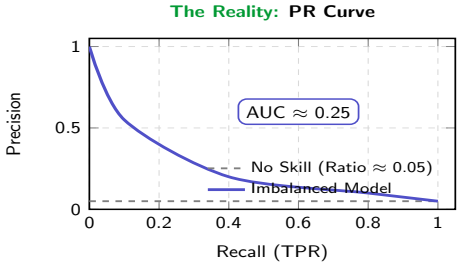
Why ROC-AUC Fails

- True Negatives dominate the False Positive Rate denominator.
- Model predicting all zeroes can hit 0.90+ AUC.

Relational Metrics

- **PR-AUC:** Focuses on the minority positive class.





The Limit of Week 2 & The Geometric Horizon

Week 1 (Tabular):

- Calculating statistics on fixed vectors.

Week 2 (Relational):

- Logic rules on dynamic sets, boosted iteratively.
- **The Bottleneck:** Tree splits are still **discrete** ($\exists$ Tx $\leq$ 1000). You cannot calculate a continuous derivative of a discrete logical literal!

Week 3 Horizon: Graph Neural Networks (GNNs)

- How do we make the *structure itself* fully differentiable?
- **Transition:** Moving from "Discrete Logic" to "Continuous Geometry".